



Proyecto Semana 08: Sistema de Gestión con Zustand



Objetivo

Implementar un sistema de gestión completo usando Zustand para el estado global, aplicando stores múltiples, selectores optimizados, y persistencia.



Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio único]

Ejemplos de Dominios

Dominio	Entidad Principal	Entidades Secundarias
Biblioteca	Book	Loan, Member
Farmacia	Medicine	Sale, Customer
Gimnasio	Member	Attendance, Plan
Escuela	Student	Course, Grade
Restaurante	Dish	Order, Table
Hospital	Patient	Appointment, Doctor
Cine	Movie	Showtime, Ticket
Hotel	Room	Booking, Guest



Requisitos Funcionales

1. Arquitectura de Stores (Obligatorio)

Crear **3 stores separados**:

Store Principal (entidad del dominio)

```
// Ejemplo: useBookStore, useMedicineStore, useMemberStore, etc.
interface MainStore {
  items: Item[];
  selectedId: number | null;
  filters: Filters;

  // CRUD
  addItem: (item:NewItem) => void;
  updateItem: (id: number, updates: Partial<Item>) => void;
  deleteItem: (id: number) => void;

  // Selección
  selectItem: (id: number | null) => void;

  // Filtros
  setFilter: (filter: Partial<Filters>) => void;
  clearFilters: () => void;
}
```

```
// Getters
getFilteredItems: () => Item[];
getSelectedItem: () => Item | undefined;
}
```

Store de UI

```
interface UIStore {
  sidebarOpen: boolean;
  theme: 'light' | 'dark';
  activeModal: string | null;
  notifications: Notification[];

  toggleSidebar: () => void;
  setTheme: (theme: 'light' | 'dark') => void;
  openModal: (modalId: string) => void;
  closeModal: () => void;
  addNotification: (notification: Notification) => void;
  removeNotification: (id: number) => void;
}
```

Store de Usuario (Auth)

```
interface AuthStore {
  user: User | null;
  isAuthenticated: boolean;
  role: 'admin' | 'user' | null;

  login: (credentials: Credentials) => Promise<void>;
  logout: () => void;
  updateProfile: (updates: Partial<User>) => void;
}
```

2. Persistencia (Obligatorio)

- Store Principal: Persistir items en localStorage
- Store de UI: Persistir tema y sidebarOpen
- Store de Auth: Persistir user y token en sessionStorage

3. Selectores Optimizados (Obligatorio)

- Usar selectores individuales (no objetos completos)
- Implementar `shallow` donde sea necesario
- Usar `useMemo` para cálculos derivados costosos

4. Comunicación Entre Stores (Obligatorio)

- Al menos una acción que acceda a múltiples stores
- Ejemplo: "Procesar operación" que verifica auth, modifica items, y notifica UI

5. DevTools (Obligatorio)

- Integrar middleware `devtools` en todos los stores
 - Nombrar acciones descriptivamente
-

Estructura del Proyecto

```
3-proyecto/
├── README.md
├── starter/
│   ├── src/
│   │   ├── stores/
│   │   │   ├── mainStore.ts      # Store principal del dominio
│   │   │   ├── uiStore.ts        # Store de UI
│   │   │   └── authStore.ts      # Store de auth
│   │   ├── components/
│   │   │   ├── ItemList.tsx
│   │   │   ├── ItemCard.tsx
│   │   │   ├── ItemForm.tsx
│   │   │   ├── Header.tsx
│   │   │   ├── Sidebar.tsx
│   │   │   └── Notifications.tsx
│   │   ├── hooks/
│   │   │   └── useFilteredItems.ts
│   │   ├── types/
│   │   │   └── index.ts
│   │   ├── App.tsx
│   │   └── main.tsx
│   ├── index.html
│   ├── package.json
│   ├── tsconfig.json
│   └── vite.config.ts
└── solution/
    └── ... (misma estructura)
```

Entregables

1. **3 stores funcionales** con TypeScript estricto
2. **Persistencia** configurada correctamente
3. **UI completa** con lista, formulario, filtros
4. **Comunicación** entre stores demostrada
5. **DevTools** funcionando
6. **README** con documentación de tu dominio

Criterios de Evaluación

Criterio	Puntos
Stores tipados correctamente	20
Persistencia funcional	15
Selectores optimizados	15
UI funcional completa	20
Comunicación entre stores	15
DevTools configurado	5

Código limpio y documentado	10
Total	100

Tips

1. **Empieza por los tipos:** Define interfaces antes de implementar
 2. **Un store a la vez:** Completa cada store antes de pasar al siguiente
 3. **Testea persistencia:** Recarga la página para verificar que los datos persisten
 4. **Usa DevTools:** Redux DevTools es tu mejor amigo para debugging
 5. **Selectores primero:** Piensa en qué datos necesita cada componente
-

Recursos

- [Zustand Docs](#)
- [Zustand Persist](#)
- [Redux DevTools](#)